

CSC240Lab8/CSC240_LinkedQueue_Practice/CSC240LinkedQueueDriverCircularTest.cpp

```
1  //=====
2  // Name      : CSC240_Queue_Linked.cpp
3  // Author     : Ivan Temesvari; Rewritten to Circular by Kazim Zaidi
4  // Version    : 9/25/2026
5  // Description : Linked Q -> Linked Q Circular; Implement Rule-of-Three
6  //=====
7
8  #include <iostream>
9  // #include "QueType.h"
10 using namespace std;
11
12 template <class ItemType>
13
14 struct NodeType
15 {
16     ItemType info;
17     NodeType<ItemType> *next;
18 };
19
20 class FullQueue
21 {
22 };
23 class EmptyQueue
24 {
25 };
26
27 template <class ItemType>
28 class QueType
29 {
30 public:
31     QueType();
32     // Class constructor.
33     // Because there is a default constructor, the precondition
34     // that the queue has been initialized is omitted.
35     QueType(int max);
36     // Parameterized class constructor.
37     ~QueType();
38     // Class destructor.
39     QueType(const QueType &anotherQue);
40     // Copy constructor
41     void MakeEmpty();
42     // Function: Initializes the queue to an empty state.
43     // Post: Queue is empty.
44     bool IsEmpty() const;
45     // Function: Determines whether the queue is empty.
46     // Post: Function value = (queue is empty)
47     bool IsFull() const;
```

```
48     // Function: Determines whether the queue is full.
49     // Post: Function value = (queue is full)
50     void Enqueue(ItemType newItem);
51     // Function: Adds newItem to the rear of the queue.
52     // Post: If (queue is full) FullQueue exception is thrown
53     //         else newItem is at rear of queue.
54     void Dequeue(ItemType &item);
55     // Function: Removes front item from the queue and returns it in item.
56     // Post: If (queue is empty) EmptyQueue exception is thrown
57     //         and item is undefined
58     //         else front element has been removed from queue and
59     //         item is a copy of removed element.
60     void Print();
61     // Function: Display the contents of the QueType in the console output.
62     // Post: The QueType remains unchanged.
63
64 private:
65     NodeType<ItemType> *rear;
66     int length;
67 };
68
69 template <class ItemType>
70 void QueType<ItemType>::Print() // rewrote to circular
71 {
72     if (isEmpty())
73     {
74         throw EmptyQueue;
75     }
76     else
77     {
78         NodeType<ItemType> *tempPtr = rear;
79         tempPtr = tempPtr->next;
80         std::cout << "Front:";
81         while (tempPtr != rear)
82         {
83             std::cout << temp->info << " ";
84             tempPtr = tempPtr->next;
85         }
86         std::cout << tempPtr->info << ":Rear" << endl;
87     }
88 }
89
90 template <class ItemType>
91
92 QueType<ItemType>::QueType() // Class constructor. // rewrote to circular
93 // Post: front and rear are set to NULL.
94 {
95     rear = nullptr;
96     length = 0;
```

```
97 }
98
99 template <class ItemType>
100
101 void QueType<ItemType>::MakeEmpty() // rewrote to circular
102
103 // Post: Queue is empty; all elements have been deallocated.
104 {
105     if (IsEmpty()) // special case 0-node
106     {
107         return; // throwing here is a poor design decision
108     }
109
110     NodeType<ItemType> *tempPtr;
111     tempPtr = rear->next; // tempPtr -> first node
112     rear->next = nullptr; // final element is marked
113
114     while (tempPtr->next != nullptr) // after while, tempPtr -> final node
115     {
116         rear = tempPtr;
117         tempPtr = tempPtr->next;
118         delete rear;
119     }
120     delete tempPtr; // final node; special case 1-node;
121     rear = nullptr; // empty queue invariant
122     length = 0;
123 }
124
125 // Class destructor.
126 template <class ItemType> // this works - no backing structure to delete after
    MakeEmpty(); no rewrite to circular
127 QueType<ItemType>::~~QueType()
128 {
129     MakeEmpty();
130 }
131
132 template <class ItemType> // no rewrite to circular
133 bool QueType<ItemType>::IsFull() const
134 // Returns true if there is no room for another ItemType
135 // on the free store; false otherwise.
136 {
137     NodeType<ItemType> *location;
138     try
139     {
140         location = new NodeType<ItemType>;
141         delete location;
142         return false;
143     }
144     catch (std::bad_alloc &)
```

```
145     {
146         return true;
147     }
148 }
149
150 template <class ItemType> // rewrote to circular
151 bool QueType<ItemType>::IsEmpty() const
152 // Returns true if there are no elements on the queue; false otherwise.
153 {
154     // return (rear == nullptr);
155     return (length == 0);
156 }
157
158 template <class ItemType>
159 void QueType<ItemType>::Enqueue(ItemType newItem) // rewrote to circular
160 // Adds newItem to the rear of the queue.
161 // Pre: Queue has been initialized.
162 // Post: If (queue is not full) newItem is at the rear of the queue;
163 //       otherwise a FullQueue exception is thrown.
164
165 {
166     if (IsFull())
167         throw FullQueue();
168     else
169     {
170         NodeType<ItemType> *newNode;
171         newNode = new NodeType<ItemType>; // pointer to newNode
172         newNode->info = newItem;
173
174         if (isEmpty())
175         {
176             rear = newNode;
177             newNode->next = rear;
178         }
179         if (!isEmpty())
180         {
181             newNode->next = rear->next;
182             rear->next = newNode;
183             rear = newNode;
184         }
185         length++;
186         // rear = newNode;
187     }
188 }
189
190 template <class ItemType> // rewrote to circular
191 void QueType<ItemType>::Dequeue(ItemType &item)
192 // Removes front item from the queue and returns it in item.
193 // Pre: Queue has been initialized and is not empty.
```

```
194 // Post: If (queue is not empty) the front of the queue has been
195 //       removed and a copy returned in item;
196 //       otherwise a EmptyQueue exception has been thrown.
197 {
198     if (IsEmpty()) // special case 0-node
199     {
200         throw EmptyQueue();
201     }
202     else if (rear->next == rear) // special case 1-node
203     {
204         delete rear;
205         rear = nullptr;
206         return;
207     }
208     else // general case n-node
209     {
210
211         NodeType<ItemType> *tempPtr;
212
213         tempPtr = rear->next;          // tempPtr points to first element
214         rear->next = rear->next->next; // last element points to second element
215         delete tempPtr;
216     }
217     length--;
218     return;
219 }
220
221 template <class ItemType> // rewrote to circular
222 QueType<ItemType>::QueType(const QueType &anotherQue)
223 {
224     NodeType<ItemType> *ptr1;
225     NodeType<ItemType> *ptr2;
226     if (anotherQue.IsEmpty()) // special case 0-node
227     {
228         length = 0;
229         rear = nullptr;
230         return;
231     }
232     else if (anotherQue.rear->next == anotherQue.rear) // special case 1-node
233     {
234         ptr1 = new NodeType<ItemType>;
235         ptr2 = anotherQue.rear;
236         ptr1->info = ptr2->info;
237         rear = ptr1;
238         length = 1;
239         return;
240     }
241     else // general case n-node
242     {
```

```
243
244     ptr2 = anotherQue.rear->next;
245     ptr1 = new NodeType<ItemType>;
246     NodeType<ItemType> *front = ptr1;
247     ptr1->info = ptr2->info;
248     for (int k = 0; k < anotherQue.length - 1; k++)
249     {
250         ptr2 = ptr2->next;
251         ptr1->next = new NodeType<ItemType>;
252         ptr1 = ptr1->next;
253         ptr1->info = ptr2->info;
254     }
255     ptr1->next = front;
256     length = anotherQue.length;
257     rear = ptr1;
258 }
259 }
260
261 int main()
262 {
263     QueType<int> q;
264     q.Enqueue(8);
265     q.Enqueue(6);
266     q.Enqueue(7);
267     q.Enqueue(5);
268     q.Enqueue(3);
269     q.Enqueue(0);
270     q.Enqueue(9);
271     q.Print();
272     int numFront;
273     q.Dequeue(numFront);
274     cout << numFront << endl;
275     q.Print();
276     return 0;
277 }
278
```